

Министерство просвещения Республики Казахстан
Колледж УО «Каспийский общественный университет»

«Допущена к защите»
Заместитель директора
по учебной работе
_____ Берденова Г.Е.
«___» _____ 2026 г.

«Утверждаю»
Директор
_____ Ануаш Ж.Д.
«___» _____ 2026 г.

ДИПЛОМНЫЙ ПРОЕКТ

**на тему: «Разработка веб-платформы для автосервиса
с системой онлайн-записи с использованием
HTML, CSS и JavaScript»**

по специальности 06130100 — Программное обеспечение (по видам)
квалификация 4S06130103 — Разработчик программного обеспечения

Выполнила

Даша Русланова

Руководитель проекта

Алматы, 2026 г.

Содержание

ВВЕДЕНИЕ	3
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ	5
1.1 Обзор индустрии автосервисных услуг в Казахстане	5
1.2 Анализ существующих решений	5
1.3 Язык программирования Python	6
1.4 Веб-фреймворк Flask	7
1.5 SQLAlchemy и базы данных	7
1.6 Технологии клиентской части: HTML, CSS и JavaScript	8
1.7 Фреймворк Bootstrap	8
1.8 Шаблонизатор Jinja2	8
1.9 Система контроля доступа и безопасность	9
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	10
2.1 Определение функциональных требований	10
2.2 Диаграмма вариантов использования	10
2.3 Архитектура приложения	11
2.4 Проектирование базы данных	12
2.4.1 Описание таблиц базы данных	13
2.4.2 ER-диаграмма базы данных	15
2.5 Проектирование маршрутов (URL-схема)	15
2.6 Жизненный цикл записи на обслуживание	16
2.7 Проектирование пользовательского интерфейса	17
2.8 Алгоритм формирования временных слотов	17
3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ПРОЕКТА	19
3.1 Инструменты разработки	19
3.2 Структура файлов проекта	19
3.3 Конфигурация приложения	20
3.4 Фабрика приложения	20
3.5 Модели базы данных	22
3.6 Система аутентификации	23
3.7 Форма записи на обслуживание	24
3.8 REST API для динамической загрузки данных	26
3.9 Административная панель	27
3.10 Декораторы доступа и вспомогательные функции	28
3.11 Формы и валидация данных	29
3.12 Инициализация базы данных и тестовые данные	29
3.13 Зависимости проекта	30
3.14 Запуск приложения	31
3.15 Тестирование приложения	31
3.16 Обеспечение безопасности приложения	33

3.17	Развёртывание приложения	34
3.18	Оптимизация производительности	35
4	ОХРАНА ТРУДА ПРИ РАБОТЕ С КОМПЬЮТЕРОМ	37
4.1	Общие положения	37
4.2	Требования к рабочему месту	37
4.3	Режим работы и отдыха	38
4.4	Электробезопасность	38
4.5	Пожарная безопасность	39
	ЗАКЛЮЧЕНИЕ	40
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
	ПРИЛОЖЕНИЯ	44
	Приложение А — Полный код файла models.py	44
	Приложение Б — Полный код файла utils.py	48
	Приложение В — Полный код файла api/routes.py	51

ВВЕДЕНИЕ

В последние годы автомобильная отрасль Казахстана переживает значительный рост. По данным Комитета по статистике Министерства национальной экономики РК, на конец 2025 года в стране зарегистрировано более 4,5 миллионов легковых автомобилей, и эта цифра ежегодно увеличивается. Вместе с ростом автопарка растёт и потребность в качественном техническом обслуживании автотранспорта. Рынок автосервисных услуг является одним из наиболее динамично развивающихся сегментов сферы обслуживания в Республике Казахстан.

Однако, несмотря на рост количества автосервисных предприятий, многие из них до сих пор работают по устаревшим моделям: запись по телефону, ведение учёта в бумажных журналах, отсутствие прозрачного ценообразования и обратной связи с клиентами. Такой подход приводит к целому ряду проблем: потеря клиентов из-за занятости телефонной линии, ошибки при записи, отсутствие возможности оперативного управления расписанием мастеров, невозможность для клиента самостоятельно выбрать удобное время обслуживания.

Цифровизация является одним из ключевых направлений стратегии «Цифровой Казахстан». Внедрение информационных технологий в сферу автосервисных услуг позволяет значительно повысить эффективность работы предприятия, улучшить качество обслуживания клиентов, автоматизировать рутинные процессы и обеспечить прозрачность ценообразования. Онлайн-запись на обслуживание стала стандартом индустрии в развитых странах и активно внедряется в Казахстане.

На рынке уже существуют зарубежные решения, такие как YCLIENTS, DIKIDI Business и AutoDealer CRM, однако они, как правило, требуют значительных финансовых затрат на лицензирование, не всегда адаптированы к специфике казахстанского рынка и зачастую избыточны для малого и среднего бизнеса. В связи с этим разработка собственной веб-платформы для автосервиса является актуальной задачей.

Цель дипломного проекта — разработка веб-платформы для автосервиса с системой онлайн-записи, которая позволит автоматизировать процесс записи клиентов на обслуживание, управление услугами, сотрудниками и расписанием работы мастеров.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. изучить предметную область и провести анализ существующих решений для автосервисных предприятий;
2. определить функциональные и нефункциональные требования к веб-платформе;
3. спроектировать архитектуру приложения и структуру базы данных;
4. реализовать серверную часть приложения с использованием фреймворка Flask;

5. разработать клиентскую часть с использованием HTML, CSS и JavaScript;
6. реализовать систему онлайн-записи с выбором услуги, мастера и временного слота;
7. провести тестирование и отладку разработанного приложения;
8. подготовить документацию к проекту.

Объект исследования — процесс записи клиентов на обслуживание в автосервисном предприятии.

Предмет исследования — автоматизация процесса онлайн-записи на обслуживание в автосервисе посредством веб-платформы.

Практическая значимость работы заключается в создании готового к эксплуатации веб-приложения, которое может быть развёрнуто в реальном автосервисном предприятии города Алматы и использоваться для автоматизации процессов записи клиентов, управления услугами и мониторинга загрузки мастеров.

Дипломный проект состоит из введения, четырёх разделов, заключения, списка использованных источников и приложений. В первом разделе рассматриваются теоретические основы разработки веб-приложений и применяемых технологий. Во втором разделе описывается проектирование системы. Третий раздел посвящён практической реализации проекта. Четвёртый раздел рассматривает вопросы охраны труда при работе за компьютером.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЙ

1.1 Обзор индустрии автосервисных услуг в Казахстане

Автосервисная отрасль Казахстана является одной из наиболее востребованных в сфере услуг. По данным аналитических исследований, средний автомобиль в Казахстане проходит техническое обслуживание от 2 до 4 раз в год, что формирует устойчивый спрос на услуги автосервисных предприятий. В крупных городах — Алматы, Астана, Шымкент — конкуренция среди автосервисов особенно высока, что заставляет предприятия искать новые способы привлечения и удержания клиентов.

Традиционно автосервисы Казахстана работали по модели телефонной записи или приёма клиентов в порядке живой очереди. Однако данная модель имеет ряд существенных недостатков:

- невозможность круглосуточной записи — клиент может позвонить только в рабочее время;
- потеря обращений из-за занятости телефонной линии;
- отсутствие визуализации доступных временных слотов;
- невозможность оперативного отслеживания загрузки мастеров;
- сложность ведения статистики и аналитики.

Внедрение цифровых технологий позволяет решить все перечисленные проблемы. Мировые тенденции показывают, что автосервисы, использующие онлайн-запись, демонстрируют рост клиентской базы на 25–40% и снижение количества пропущенных записей на 50–60%.

1.2 Анализ существующих решений

На рынке представлено несколько платформ, предназначенных для автоматизации работы сервисных предприятий. Рассмотрим наиболее популярные из них.

YCLIENTS (yclients.com) — российская облачная платформа для онлайн-записи, которая широко используется в сфере услуг. Платформа предоставляет виджет записи, CRM-систему, аналитику, SMS-напоминания. Среди недостатков — высокая стоимость подписки (от 5000 рублей в месяц), ориентированность преимущественно на салоны красоты и медицинские учреждения, а не на автосервисы.

DIKIDI Business — ещё одна российская платформа для онлайн-записи, которая также адаптирована под различные виды услуг. Преимуществом является наличие мобильного приложения и бесплатного базового тарифа. К недостаткам относится ограниченная функциональность в бесплатной версии и отсутствие специализированных модулей для автосервисов (учёт транспортных средств, VIN-номера, пробег).

AutoDealer CRM — специализированная CRM-система для автодилеров

и автосервисов. Система включает в себя управление заказ-нарядами, складской учёт запчастей, ведение истории обслуживания автомобилей. Однако данное решение является громоздким и дорогостоящим, что делает его недоступным для небольших автосервисных предприятий.

Сравнительный анализ существующих решений представлен в таблице 1.

Таблица 1 – Сравнительный анализ существующих решений

Критерий	YCLIENTS	DIKIDI	AutoDealer	Наш проект
Онлайн-запись	Да	Да	Да	Да
Выбор мастера	Да	Да	Нет	Да
Временные слоты	Да	Да	Нет	Да
Отзывы/рейтинги	Нет	Частично	Нет	Да
Учёт авто	Нет	Нет	Да	Да
Стоимость	Платная	Условно-бесп.	Платная	Бесплатная
Open source	Нет	Нет	Нет	Да
Адаптация КЗ	Частично	Частично	Нет	Да

На основе проведённого анализа можно сделать вывод, что существующие решения либо не адаптированы под специфику автосервисных предприятий, либо имеют высокую стоимость, либо не доступны для кастомизации. Это подтверждает целесообразность разработки собственного решения.

1.3 Язык программирования Python

Python — высокоуровневый интерпретируемый язык программирования общего назначения, созданный Гвидо ван Россумом и впервые выпущенный в 1991 году. На сегодняшний день Python является одним из наиболее популярных языков программирования в мире, занимая первые позиции в рейтингах TIOBE и Stack Overflow Developer Survey.

Основные преимущества Python для веб-разработки:

- простой и читаемый синтаксис, что ускоряет процесс разработки;
- обширная стандартная библиотека и множество сторонних пакетов;
- поддержка объектно-ориентированного программирования;
- наличие мощных веб-фреймворков (Django, Flask, FastAPI);
- активное сообщество разработчиков и обширная документация;
- кроссплатформенность — работает на Windows, macOS и Linux.

Для данного дипломного проекта был выбран Python версии 3.x в связке с микро-фреймворком Flask, что обеспечивает баланс между функциональностью и простотой разработки.

1.4 Веб-фреймворк Flask

Flask — это микро-фреймворк для разработки веб-приложений на языке Python, созданный Армином Ронахером. Микро-фреймворк означает, что Flask предоставляет минимальный набор инструментов и позволяет разработчику самостоятельно выбирать необходимые расширения.

Ключевые особенности Flask:

- лёгкий и минималистичный — ядро фреймворка содержит только маршрутизацию и систему шаблонов Jinja2;
- модульная архитектура с использованием Blueprints для организации кода;
- встроенный сервер разработки с автоматической перезагрузкой;
- поддержка WSGI-совместимых серверов для развёртывания;
- богатая экосистема расширений: Flask-SQLAlchemy, Flask-Login, Flask-WTF, Flask-Migrate и другие;
- встроенный механизм сессий и работа с cookie.

В дипломном проекте используется Flask версии 3.0.3 совместно со следующими расширениями:

- **Flask-SQLAlchemy 3.1.1** — ORM для работы с базой данных;
- **Flask-Login 0.6.3** — управление аутентификацией пользователей;
- **Flask-WTF 1.2.1** — работа с HTML-формами и CSRF-защита;
- **Flask-Migrate 4.0.7** — миграции базы данных.

1.5 SQLAlchemy и базы данных

SQLAlchemy — это библиотека для работы с реляционными базами данных в Python, предоставляющая полнофункциональный ORM (Object-Relational Mapping). ORM позволяет работать с таблицами базы данных как с объектами Python, абстрагируясь от написания SQL-запросов вручную.

В качестве системы управления базами данных в проекте используется SQLite — встраиваемая реляционная СУБД, которая не требует отдельного серверного процесса и хранит всю базу данных в одном файле. SQLite идеально подходит для приложений малого и среднего масштаба, обеспечивая при этом поддержку стандарта SQL, транзакции и ACID-совместимость.

Основные преимущества связки SQLAlchemy + SQLite:

- абстракция работы с базой данных через Python-объекты;
- автоматическое создание таблиц по описанию моделей;
- поддержка связей между таблицами (one-to-many, many-to-many);
- встроенная валидация данных на уровне модели;
- простота развёртывания — не требуется установка СУБД;
- возможность миграции на PostgreSQL или MySQL при масштабировании.

1.6 Технологии клиентской части: HTML, CSS и JavaScript

Клиентская часть веб-приложения разрабатывается с использованием стандартных технологий веб-разработки.

HTML (HyperText Markup Language) — язык разметки гипертекста, являющийся основой структуры любой веб-страницы. В проекте используется HTML5, который предоставляет семантические элементы (header, footer, nav, section, article), встроенные формы с валидацией, поддержку мультимедиа и улучшенную доступность.

CSS (Cascading Style Sheets) — каскадные таблицы стилей, определяющие визуальное оформление веб-страниц. CSS3 обеспечивает гибкую раскладку (Flexbox, Grid), адаптивный дизайн через медиа-запросы, анимации и переходы, кастомные свойства (CSS-переменные).

JavaScript — язык программирования, обеспечивающий интерактивность веб-страниц. В проекте JavaScript используется для динамического обновления содержимого без перезагрузки страницы (AJAX-запросы к API), валидации форм на стороне клиента, реализации интерактивного выбора временных слотов и работы с рейтингом в виде звёзд.

1.7 Фреймворк Bootstrap

Bootstrap — популярный CSS-фреймворк для создания адаптивных веб-интерфейсов, разработанный компанией Twitter. В дипломном проекте используется Bootstrap 5, который предоставляет:

- готовую сетку из 12 колонок с контрольными точками адаптивности;
- обширную библиотеку компонентов: кнопки, карточки, модальные окна, навигационные панели, таблицы, бейджи, алерты;
- утилитарные классы для быстрой стилизации элементов;
- иконки Font Awesome для визуального оформления элементов интерфейса;
- адаптивный дизайн «из коробки» для корректного отображения на мобильных устройствах.

Использование Bootstrap позволяет значительно ускорить процесс разработки пользовательского интерфейса и обеспечить единообразное отображение на различных устройствах и в разных браузерах.

1.8 Шаблонизатор Jinja2

Jinja2 — это мощный и гибкий шаблонизатор для Python, который используется Flask по умолчанию. Jinja2 позволяет разделять логику приложения и представление, следуя принципу MVC (Model-View-Controller).

Основные возможности Jinja2:

- наследование шаблонов через механизм extends/block;
- условные конструкции и циклы внутри шаблонов;
- пользовательские фильтры для форматирования данных;

- автоматическое экранирование HTML для предотвращения XSS-атак;
- макросы для повторного использования фрагментов разметки.

В проекте используется наследование шаблонов: базовый шаблон `base.html` содержит общую структуру страницы (навигация, подключение CSS/JS, подвал), а дочерние шаблоны наследуют его и переопределяют блоки содержимого.

1.9 Система контроля доступа и безопасность

Безопасность веб-приложения обеспечивается комплексом мер:

- **Хеширование паролей** — пароли хранятся в базе данных в виде хешей, генерируемых с использованием библиотеки Werkzeug (функции `generate_password_hash` и `check_password_hash`);
- **CSRF-защита** — все формы защищены от атак Cross-Site Request Forgery с использованием Flask-WTF и токенов;
- **Аутентификация** — реализована с помощью Flask-Login, которая управляет сессиями пользователей, обеспечивает функционал «Запомнить меня» и перенаправление неавторизованных пользователей;
- **Авторизация** — реализована ролевая модель доступа с тремя ролями: администратор, сотрудник и клиент. Каждая роль имеет строго определённый набор разрешений;
- **Валидация данных** — данные валидируются как на стороне клиента (JavaScript), так и на стороне сервера (WTForms).

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Определение функциональных требований

На основе анализа предметной области и существующих решений были сформулированы функциональные требования к веб-платформе. Требования разделены по ролям пользователей.

Требования для роли «Клиент»:

1. регистрация и авторизация в системе;
2. просмотр каталога услуг с ценами и описаниями;
3. фильтрация услуг по категориям и поиск;
4. просмотр профилей мастеров с рейтингами;
5. онлайн-запись на обслуживание с выбором услуги, мастера, даты и времени;
6. указание информации об автомобиле и дополнительных пожеланий;
7. просмотр истории записей и их статусов;
8. отмена записи в статусе «Ожидает» или «Подтверждено»;
9. оставление отзыва и оценки после завершения обслуживания;
10. редактирование профиля и смена пароля.

Требования для роли «Администратор»:

1. панель управления с основными показателями (дашборд);
2. управление категориями услуг (создание, редактирование, удаление);
3. управление услугами (CRUD-операции, загрузка изображений);
4. управление сотрудниками (профили, расписание, назначение услуг);
5. управление записями (просмотр, фильтрация, изменение статуса);
6. управление клиентами (просмотр, блокировка/разблокировка);
7. модерация отзывов (публикация, скрытие, удаление);
8. просмотр статистики: выручка, количество записей, рейтинги.

Нефункциональные требования:

- адаптивный дизайн для корректного отображения на мобильных устройствах;
- время загрузки страницы не более 3 секунд;
- поддержка одновременной работы нескольких пользователей;
- защита персональных данных пользователей;
- интуитивно понятный интерфейс, не требующий обучения.

2.2 Диаграмма вариантов использования

Для визуализации взаимодействия пользователей с системой была разработана диаграмма вариантов использования (Use Case Diagram) в нотации UML.



Рисунок 1 – Диаграмма вариантов использования

2.3 Архитектура приложения

Приложение построено по архитектурному паттерну MVC (Model-View-Controller) с использованием модульной структуры Flask Blueprints. Данный подход позволяет разделить код приложения на логически независимые модули, каждый из которых отвечает за свою область функциональности.

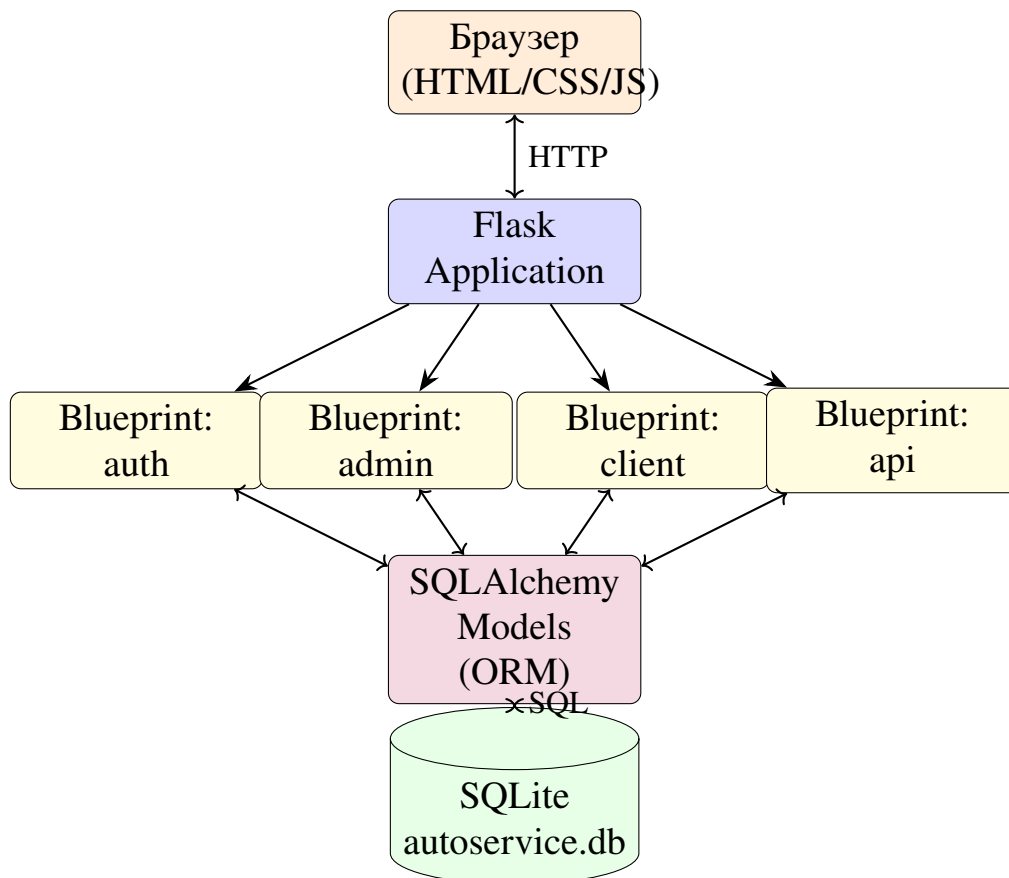


Рисунок 2 – Архитектура приложения

Приложение состоит из следующих модулей (Blueprints):

- **auth** — модуль аутентификации (регистрация, вход, выход, профиль, смена пароля);
- **admin** — модуль административной панели (дашборд, управление услугами, сотрудниками, записями, клиентами, отзывами);
- **client** — модуль клиентской части (главная страница, каталог услуг, запись на обслуживание, мои записи, мастера);
- **api** — модуль REST API (получение доступных временных слотов, списка мастеров для услуги, событий для календаря).

2.4 Проектирование базы данных

База данных является ключевым компонентом системы. В результате проектирования была разработана реляционная схема, состоящая из 7 основных таблиц и 1 связующей таблицы.

2.4.1 Описание таблиц базы данных

Таблица 2 – Таблица users — пользователи системы

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
email	String(120)	UQ, IDX	Электронная почта
password_hash	String(256)		Хеш пароля
role	String(20)		Роль: admin/employee/client
first_name	String(50)		Имя пользователя
last_name	String(50)		Фамилия пользователя
phone	String(20)		Номер телефона
avatar	String(255)		Путь к аватару
is_active	Boolean		Флаг активности
created_at	DateTime		Дата регистрации

Таблица 3 – Таблица service_categories — категории услуг

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
name	String(100)		Название категории
description	Text		Описание категории
icon	String(50)		CSS-класс иконки
slug	String(100)	UQ	URL-адрес категории
is_active	Boolean		Флаг активности
sort_order	Integer		Порядок сортировки

Таблица 4 – Таблица services — услуги автосервиса

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
name	String(200)		Название услуги
description	Text		Полное описание
short_description	String(300)		Краткое описание
price	Numeric(10,2)		Цена в тенге
duration	Integer		Длительность в минутах
category_id	Integer	FK	Ссылка на категорию
image	String(255)		Путь к изображению
is_active	Boolean		Флаг активности
is_featured	Boolean		Рекомендуемая услуга
created_at	DateTime		Дата создания

Таблица 5 – Таблица employees — профили сотрудников

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
user_id	Integer	FK, UQ	Ссылка на пользователя
specialization	String(200)		Специализация мастера
bio	Text		Биография мастера
experience_years	Integer		Стаж работы (лет)
photo	String(255)		Путь к фотографии
is_available	Boolean		Доступен для записи

Таблица 6 – Таблица work_schedules — расписание работы

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
employee_id	Integer	FK	Ссылка на сотрудника
day_of_week	Integer		День недели (0–6)
start_time	Time		Начало рабочего дня
end_time	Time		Конец рабочего дня
is_working	Boolean		Флаг рабочего дня

Таблица 7 – Таблица bookings — записи на обслуживание

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
booking_number	String(20)	UQ	Номер записи (AS...)
client_id	Integer	FK	Ссылка на клиента
employee_id	Integer	FK	Ссылка на мастера
service_id	Integer	FK	Ссылка на услугу
booking_date	Date		Дата обслуживания
start_time	Time		Время начала
end_time	Time		Время окончания
status	String(20)		Статус записи
total_price	Numeric(10,2)		Итоговая цена
car_info	String(200)		Информация об авто
notes	Text		Примечания клиента
admin_notes	Text		Заметки администратора
created_at	DateTime		Дата создания
updated_at	DateTime		Дата обновления

Таблица 8 – Таблица reviews — отзывы клиентов

Поле	Тип	Ключ	Описание
id	Integer	PK	Уникальный идентификатор
booking_id	Integer	FK, UQ	Ссылка на запись
client_id	Integer	FK	Ссылка на клиента
rating	Integer		Оценка (1–5)
comment	Text		Текст отзыва
is_published	Boolean		Флаг публикации
created_at	DateTime		Дата создания

Дополнительно создана связующая таблица employee_services для реализации связи «многие-ко-многим» между сотрудниками и услугами. Она содержит два поля: employee_id (FK на employees) и service_id (FK на services).

2.4.2 ER-диаграмма базы данных

Связи между таблицами представлены на рисунке 3.

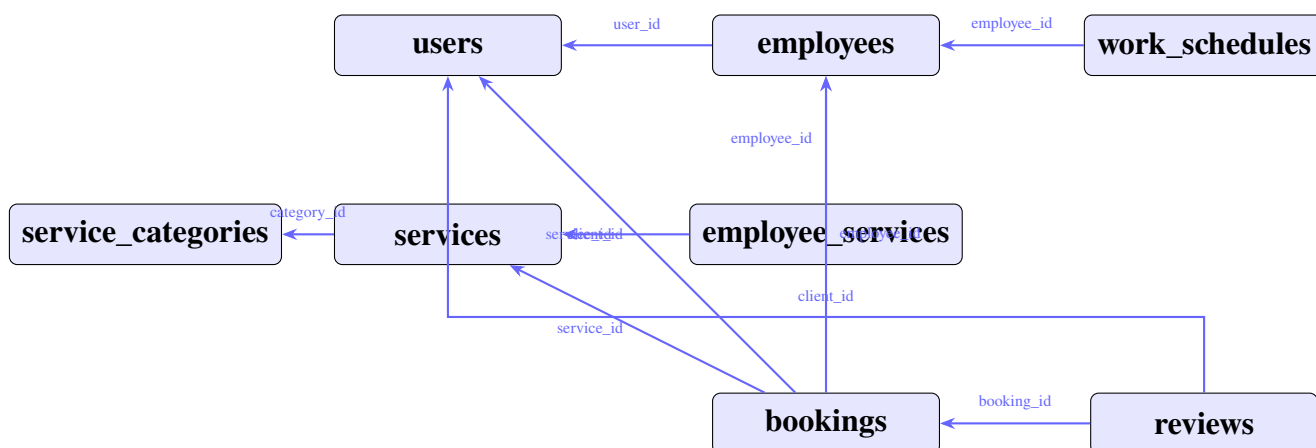


Рисунок 3 – ER-диаграмма базы данных

2.5 Проектирование маршрутов (URL-схема)

Маршрутизация приложения организована по модулям (Blueprints). Каждый модуль имеет свой префикс URL.

Таблица 9 – Основные маршруты модуля auth

URL	Методы	Описание
/auth/login	GET, POST	Страница входа в систему
/auth/register	GET, POST	Страница регистрации
/auth/logout	GET	Выход из системы
/auth/profile	GET, POST	Профиль пользователя

Таблица 10 – Основные маршруты модуля client

URL	Методы	Описание
/	GET	Главная страница
/services	GET	Каталог услуг
/services/<id>	GET	Детальная страница услуги
/booking	GET, POST	Форма записи на обслуживание
/booking/success/<id>	GET	Подтверждение записи
/my-bookings	GET	Список записей клиента
/my-bookings/<id>	GET	Детали записи
/my-bookings/<id>/cancel	POST	Отмена записи
/my-bookings/<id>/review	POST	Оставить отзыв
/employees	GET	Список мастеров
/about	GET	Страница «О нас»
/contacts	GET	Страница контактов

Таблица 11 – Основные маршруты модуля admin

URL	Методы	Описание
/admin/	GET	Панель управления (дашборд)
/admin/bookings	GET	Список всех записей
/admin/bookings/<id>	GET, POST	Детали и статус записи
/admin/services	GET	Список услуг
/admin/services/new	GET, POST	Создание услуги
/admin/services/<id>/edit	GET, POST	Редактирование услуги
/admin/categories	GET	Список категорий
/admin/employees	GET	Список сотрудников
/admin/employees/new	GET, POST	Добавление сотрудника
/admin/employees/<id>/schedule	GET, POST	Расписание сотрудника
/admin/clients	GET	Список клиентов
/admin/reviews	GET	Модерация отзывов

Таблица 12 – Маршруты REST API

URL	Методы	Описание
/api/available-slots	GET	Получение свободных слотов
/api/employees-for-service	GET	Мастера для услуги
/api/calendar-events	GET	События для календаря

2.6 Жизненный цикл записи на обслуживание

Запись на обслуживание проходит через несколько статусов, образуя конечный автомат состояний:

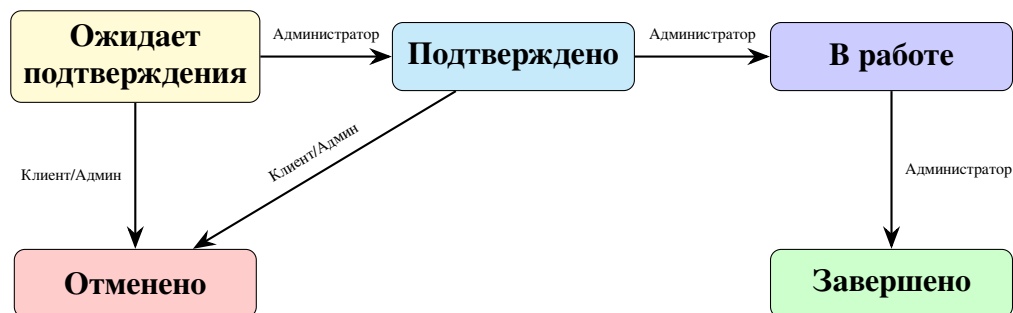


Рисунок 4 – Диаграмма состояний записи

2.7 Проектирование пользовательского интерфейса

Пользовательский интерфейс веб-платформы спроектирован с учётом следующих принципов:

- **Простота навигации** — главное меню содержит ссылки на основные разделы: «Услуги», «Мастера», «Запись», «О нас», «Контакты»;
- **Адаптивная вёрстка** — интерфейс корректно отображается на экранах от 320px (мобильные устройства) до 1920px (настольные мониторы);
- **Единая цветовая схема** — основные цвета: синий (#0dbefd) для акцентных элементов, белый и светло-серый (#f8f9fa) для фона, тёмно-серый (#212529) для текста;
- **Карточная компоновка** — услуги, мастера и записи отображаются в виде карточек Bootstrap Cards;
- **Пошаговая форма записи** — клиент последовательно выбирает услугу, мастера, дату, временной слот и подтверждает запись;
- **Информативная обратная связь** — flash-сообщения уведомляют пользователя о результате действий.

Для административной панели используется отдельный макет с боковой навигационной панелью, содержащей ссылки на все разделы управления: дашборд, записи, услуги, категории, сотрудники, клиенты, отзывы.

2.8 Алгоритм формирования временных слотов

Ключевым элементом системы онлайн-записи является алгоритм формирования доступных временных слотов. Алгоритм работает следующим образом:

1. Клиент выбирает услугу, мастера и дату;
2. Система определяет расписание мастера на выбранный день недели;
3. Если мастер не работает в этот день — возвращается пустой список;
4. Загружаются все существующие записи мастера на эту дату (со статусами pending, confirmed, in_progress);
5. Формируется множество «занятых» слотов с шагом 30 минут;
6. Генерируются все возможные слоты от начала до конца рабочего дня с шагом 30 минут;
7. Для каждого потенциального слота проверяется, что все 30-минутные

интервалы, необходимые для услуги, свободны;

8. Возвращается список доступных слотов в формате JSON.

Данный алгоритм учитывает длительность услуги: если услуга занимает 90 минут, то для неё необходимо 3 последовательных свободных 30-минутных слота.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ПРОЕКТА

3.1 Инструменты разработки

Для реализации дипломного проекта использовались следующие инструменты:

- **Операционная система:** Windows 10/11 (совместимо с macOS и Linux);
- **Среда разработки:** Visual Studio Code с расширениями Python, Jinja2;
- **Язык программирования:** Python 3.12;
- **Веб-фреймворк:** Flask 3.0.3;
- **ORM:** SQLAlchemy 2.0.30;
- **СУБД:** SQLite 3;
- **Фронтенд:** HTML5, CSS3, JavaScript (ES6), Bootstrap 5;
- **Управление зависимостями:** pip, requirements.txt;
- **Версионный контроль:** Git;
- **Браузеры для тестирования:** Google Chrome, Mozilla Firefox.

3.2 Структура файлов проекта

Файловая структура проекта организована по модульному принципу:

Листинг 1: Файловая структура проекта

```
1 autoservice/  
2   run.py                # Точка входа  
3   config.py             # Конфигурация  
4   init_db.py            # Инициализация БД  
5   requirements.txt      # Зависимости  
6   autoservice.db        # База данных SQLite  
7   app/  
8       __init__.py       # Фабрика приложения  
9       models.py         # Модели базы данных  
10      utils.py           # Вспомогательные функции  
11      auth/  
12          __init__.py    # Blueprint auth  
13          forms.py       # Формы авторизации  
14          routes.py      # Маршруты авторизации  
15      admin/  
16          __init__.py    # Blueprint admin  
17          forms.py       # Формы админ-панели  
18          routes.py      # Маршруты админ-панели  
19      client/  
20          __init__.py    # Blueprint client  
21          forms.py       # Формы клиента  
22          routes.py      # Маршруты клиента  
23      api/  
24          __init__.py    # Blueprint api  
25          routes.py      # REST API маршруты  
26      templates/  
27          base.html      # HTML-шаблон (Jinja2)  
28          auth/
```

```

29         admin/
30         client/
31     static/                # Статические файлы
32         css/
33         js/
34         uploads/

```

3.3 Конфигурация приложения

Конфигурация приложения вынесена в отдельный класс Config в файле config.py:

Листинг 2: Конфигурация приложения (config.py)

```

1  import os
2  from datetime import timedelta
3
4  basedir = os.path.abspath(os.path.dirname(__file__))
5
6  class Config:
7      SECRET_KEY = os.environ.get('SECRET_KEY') or \
8          'autoservice-diploma-secret-2024-xK9mP'
9      SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL') or \
10         'sqlite:/// ' + os.path.join(basedir, 'autoservice.db')
11      SQLALCHEMY_TRACK_MODIFICATIONS = False
12      PERMANENT_SESSION_LIFETIME = timedelta(days=7)
13
14      UPLOAD_FOLDER = os.path.join(basedir, 'app',
15                                  'static', 'uploads')
16      MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16MB
17      ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg',
18                           'gif', 'webp'}
19
20      # Business settings
21      BUSINESS_NAME = 'АвтоСервис Про Алматы'
22      BUSINESS_START_HOUR = 8
23      BUSINESS_END_HOUR = 20
24      SLOT_DURATION_MINUTES = 30
25      WTF_CSRF_ENABLED = True

```

Конфигурация включает секретный ключ для подписи сессий, URI базы данных, параметры загрузки файлов (до 16 МБ), а также бизнес-настройки: название автосервиса, рабочие часы (с 8:00 до 20:00) и длительность временного слота (30 минут).

3.4 Фабрика приложения

Flask использует паттерн «фабрика приложения» (Application Factory), который позволяет создавать экземпляр приложения с различными конфигурациями:

Листинг 3: Фабрика приложения (app/__init__.py)

```

1 from flask import Flask
2 from flask_login import LoginManager
3 from flask_migrate import Migrate
4 from flask_wtf.csrf import CSRFProtect, generate_csrf
5 from config import Config
6 from app.models import db, User
7
8 login_manager = LoginManager()
9 migrate = Migrate()
10 csrf = CSRFProtect()
11
12 def create_app(config_class=Config):
13     app = Flask(__name__)
14     app.config.from_object(config_class)
15
16     # Init extensions
17     db.init_app(app)
18     login_manager.init_app(app)
19     migrate.init_app(app, db)
20     csrf.init_app(app)
21
22     login_manager.login_view = 'auth.login'
23     login_manager.login_message = \
24         'Пожалуйста, войдите в систему.'
25     login_manager.login_message_category = 'warning'
26
27     @login_manager.user_loader
28     def load_user(user_id):
29         return User.query.get(int(user_id))
30
31     # Register blueprints
32     from app.auth import bp as auth_bp
33     app.register_blueprint(auth_bp, url_prefix='/auth')
34
35     from app.admin import bp as admin_bp
36     app.register_blueprint(admin_bp, url_prefix='/admin')
37
38     from app.client import bp as client_bp
39     app.register_blueprint(client_bp)
40
41     from app.api import bp as api_bp
42     app.register_blueprint(api_bp, url_prefix='/api')
43
44     return app

```

Фабрика инициализирует все расширения Flask (SQLAlchemy, LoginManager, Migrate, CSRFProtect), регистрирует четыре Blueprint-модуля с соответствующими префиксами URL, а также определяет функцию загрузки пользователя для системы аутентификации.

3.5 Модели базы данных

Модели базы данных описаны в файле `app/models.py` с использованием SQLAlchemy ORM. Рассмотрим ключевые модели.

Листинг 4: Модель пользователя (User)

```
1 class User(UserMixin, db.Model):
2     __tablename__ = 'users'
3
4     id = db.Column(db.Integer, primary_key=True)
5     email = db.Column(db.String(120), unique=True,
6                       nullable=False, index=True)
7     password_hash = db.Column(db.String(256),
8                               nullable=False)
9     role = db.Column(db.String(20), nullable=False,
10                     default='client')
11     first_name = db.Column(db.String(50), nullable=False)
12     last_name = db.Column(db.String(50), nullable=False)
13     phone = db.Column(db.String(20))
14     avatar = db.Column(db.String(255),
15                       default='default_avatar.png')
16     is_active = db.Column(db.Boolean, default=True)
17     created_at = db.Column(db.DateTime,
18                           default=datetime.utcnow)
19
20     def set_password(self, password):
21         self.password_hash = generate_password_hash(
22             password)
23
24     def check_password(self, password):
25         return check_password_hash(
26             self.password_hash, password)
27
28     @property
29     def full_name(self):
30         return f"{self.first_name} {self.last_name}"
31
32     def is_admin(self):
33         return self.role == 'admin'
```

Модель `User` наследуется от `UserMixin` (Flask-Login) и `db.Model` (SQLAlchemy). Пароль хранится в виде хеша, что обеспечивает безопасность. Роль пользователя определяется полем `role`, которое принимает одно из значений: `admin`, `employee` или `client`.

Листинг 5: Модель записи на обслуживание (Booking)

```
1 class Booking(db.Model):
2     __tablename__ = 'bookings'
3
4     STATUS_PENDING = 'pending'
5     STATUS_CONFIRMED = 'confirmed'
```

```

6 STATUS_IN_PROGRESS = 'in_progress'
7 STATUS_COMPLETED = 'completed'
8 STATUS_CANCELLED = 'cancelled'
9
10 STATUS_CHOICES = {
11     STATUS_PENDING: 'Ожидает подтверждения',
12     STATUS_CONFIRMED: 'Подтверждено',
13     STATUS_IN_PROGRESS: 'В работе',
14     STATUS_COMPLETED: 'Завершено',
15     STATUS_CANCELLED: 'Отменено'
16 }
17
18 id = db.Column(db.Integer, primary_key=True)
19 booking_number = db.Column(db.String(20),
20                             unique=True)
21 client_id = db.Column(db.Integer,
22                        db.ForeignKey('users.id'), nullable=False)
23 employee_id = db.Column(db.Integer,
24                          db.ForeignKey('employees.id'), nullable=False)
25 service_id = db.Column(db.Integer,
26                         db.ForeignKey('services.id'), nullable=False)
27 booking_date = db.Column(db.Date, nullable=False)
28 start_time = db.Column(db.Time, nullable=False)
29 end_time = db.Column(db.Time, nullable=False)
30 status = db.Column(db.String(20),
31                    default=STATUS_PENDING)
32 total_price = db.Column(db.Numeric(10, 2),
33                          nullable=False)
34 car_info = db.Column(db.String(200))
35 notes = db.Column(db.Text)
36
37 def can_cancel(self):
38     return self.status in (self.STATUS_PENDING,
39                             self.STATUS_CONFIRMED)
40
41 def can_review(self):
42     return (self.status == self.STATUS_COMPLETED
43             and not self.review)

```

Модель Booking хранит всю информацию о записи: клиент, мастер, услуга, дата и время, статус, цена, информация об автомобиле. Методы `can_cancel()` и `can_review()` реализуют бизнес-логику управления записью.

3.6 Система аутентификации

Аутентификация пользователей реализована в модуле `auth`. Рассмотрим обработчик входа в систему:

Листинг 6: Обработчик входа в систему (`auth/routes.py`)

```

1 @bp.route('/login', methods=['GET', 'POST'])
2 def login():

```



```

3     if current_user.is_authenticated:
4         return redirect(url_for('client.index'))
5     form = LoginForm()
6     if form.validate_on_submit():
7         user = User.query.filter_by(
8             email=form.email.data.lower()).first()
9         if user and user.check_password(
10             form.password.data):
11             if not user.is_active:
12                 flash('Ваш аккаунт заблокирован.',
13                     'danger')
14                 return redirect(url_for('auth.login'))
15             login_user(user,
16                         remember=form.remember_me.data)
17             next_page = request.args.get('next')
18             if user.is_admin():
19                 return redirect(next_page or
20                     url_for('admin.dashboard'))
21             return redirect(next_page or
22                 url_for('client.index'))
23             flash('Неверный email или пароль.', 'danger')
24     return render_template('auth/login.html',
25                           title='Вход', form=form)

```

Обработчик проверяет, не авторизован ли пользователь уже, валидирует форму, ищет пользователя по email, проверяет пароль и статус активности. После успешной аутентификации администратор перенаправляется в панель управления, а клиент — на главную страницу.

3.7 Форма записи на обслуживание

Процесс записи является центральной функцией приложения. Форма записи позволяет клиенту выбрать услугу, мастера, дату и доступный временной слот:

Листинг 7: Обработка формы записи (client/routes.py)

```

1 @bp.route('/booking', methods=['GET', 'POST'])
2 @login_required
3 def booking():
4     service_id = request.args.get('service_id',
5                                   type=int)
6     service = None
7     if service_id:
8         service = Service.query.filter_by(
9             id=service_id, is_active=True
10         ).first_or_404()
11
12     form = BookingForm()
13
14     if service:
15         available_employees = [

```

```

16         e for e in service.employees
17         if e.is_available]
18     else:
19         available_employees = Employee.query.filter_by(
20             is_available=True).all()
21
22     form.employee_id.choices = [
23         (e.id, f"{e.user.full_name} - "
24             f"{e.specialization or 'Мастер'}")
25         for e in available_employees]
26
27     if form.validate_on_submit():
28         emp = Employee.query.get(form.employee_id.data)
29         svc = Service.query.get(
30             int(form.service_id.data))
31
32         booking_date = form.booking_date.data
33         start_time_obj = datetime.strptime(
34             form.start_time.data, '%H:%M').time()
35
36         if not is_slot_available(emp.id, booking_date,
37             start_time_obj, svc.duration):
38             flash('Выбранное время уже занято.',
39                 'warning')
40             return redirect(url_for('client.booking',
41                 service_id=service_id))
42
43         end_time_obj = (datetime.combine(
44             booking_date, start_time_obj) +
45             timedelta(minutes=svc.duration)).time()
46
47         new_booking = Booking(
48             booking_number=generate_booking_number(),
49             client_id=current_user.id,
50             employee_id=emp.id,
51             service_id=svc.id,
52             booking_date=booking_date,
53             start_time=start_time_obj,
54             end_time=end_time_obj,
55             total_price=svc.price,
56             car_info=form.car_info.data,
57             notes=form.notes.data,
58             status=Booking.STATUS_PENDING
59         )
60         db.session.add(new_booking)
61         db.session.commit()
62         flash('Запись успешно создана!', 'success')
63         return redirect(url_for(
64             'client.booking_success',
65             booking_id=new_booking.id))
66

```

```

67     return render_template('client/booking.html',
68                            title='Запись на услугу', form=form,
69                            service=service,
70                            available_employees=available_employees)

```

Данный обработчик выполняет следующие действия: загружает выбранную услугу, формирует список доступных мастеров, при отправке формы проверяет доступность слота, рассчитывает время окончания обслуживания, создаёт новую запись в базе данных и перенаправляет клиента на страницу подтверждения.

3.8 REST API для динамической загрузки данных

Модуль API предоставляет эндпоинты для AJAX-запросов с клиентской стороны. Ключевой эндпоинт — получение доступных временных слотов:

Листинг 8: API получения доступных слотов (api/routes.py)

```

1  @bp.route('/available-slots')
2  @login_required
3  def available_slots():
4      employee_id = request.args.get('employee_id',
5                                     type=int)
6      service_id = request.args.get('service_id',
7                                    type=int)
8      date_str = request.args.get('date', '')
9
10     if not all([employee_id, service_id, date_str]):
11         return jsonify({'error': 'Missing params'}), 400
12
13     target_date = datetime.strptime(
14         date_str, '%Y-%m-%d').date()
15
16     if target_date < date.today():
17         return jsonify({'slots': []})
18
19     service = Service.query.get(service_id)
20     schedule = WorkSchedule.query.filter_by(
21         employee_id=employee_id,
22         day_of_week=target_date.weekday(),
23         is_working=True).first()
24
25     if not schedule:
26         return jsonify({'slots': [],
27                         'message': 'Мастер не работает'})
28
29     busy = get_busy_slots(employee_id, target_date,
30                           service.duration)
31
32     slots = []
33     current = datetime.combine(target_date,
34                                schedule.start_time)

```

```

35     schedule_end = datetime.combine(target_date,
36                                     schedule.end_time)
37     service_delta = timedelta(
38         minutes=service.duration)
39     slot_step = timedelta(minutes=30)
40
41     while current + service_delta <= schedule_end:
42         is_free = True
43         check = current
44         while check < current + service_delta:
45             if check.time() in busy:
46                 is_free = False
47                 break
48             check += slot_step
49         if is_free:
50             slots.append(
51                 current.time().strftime('%H:%M'))
52             current += slot_step
53
54     return jsonify({'slots': slots})

```

API возвращает данные в формате JSON. JavaScript на стороне клиента выполняет AJAX-запрос при выборе мастера и даты, получает список доступных слотов и динамически отображает их в виде кнопок для выбора.

3.9 Административная панель

Административная панель предоставляет полный набор инструментов для управления автосервисом. Главная страница панели (дашборд) отображает ключевые показатели:

Листинг 9: Дашборд администратора (admin/routes.py)

```

1 @bp.route('/dashboard')
2 @login_required
3 @admin_required
4 def dashboard():
5     today = date.today()
6     month_start = today.replace(day=1)
7
8     stats = {
9         'total_bookings': Booking.query.count(),
10        'today_bookings': Booking.query.filter_by(
11            booking_date=today).count(),
12        'month_bookings': Booking.query.filter(
13            Booking.booking_date >= month_start
14        ).count(),
15        'pending_bookings': Booking.query.filter_by(
16            status='pending').count(),
17        'total_clients': User.query.filter_by(
18            role='client').count(),
19        'total_employees': Employee.query.count(),

```

```

20         'total_services': Service.query.filter_by(
21             is_active=True).count(),
22         'total_revenue': db.session.query(
23             func.sum(Booking.total_price)
24         ).filter_by(status='completed').scalar() or 0,
25     }
26
27     recent_bookings = (Booking.query
28         .order_by(Booking.created_at.desc())
29         .limit(8).all())
30
31     today_bookings = (Booking.query
32         .filter_by(booking_date=today)
33         .order_by(Booking.start_time).all())
34
35     return render_template('admin/dashboard.html',
36         title='Панель управления', stats=stats,
37         recent_bookings=recent_bookings,
38         today_bookings=today_bookings)

```

Дашборд агрегирует данные о количестве записей (общее, за сегодня, за месяц, ожидающих), количестве клиентов, сотрудников и услуг, общей и месячной выручке. Также отображаются таблица последних записей и расписание на текущий день.

3.10 Декораторы доступа и вспомогательные функции

Для контроля доступа реализованы декораторы, ограничивающие доступ к маршрутам:

Листинг 10: Декоратор проверки прав администратора (utils.py)

```

1 def admin_required(f):
2     @wraps(f)
3     def decorated_function(*args, **kwargs):
4         if not current_user.is_authenticated \
5             or not current_user.is_admin():
6             abort(403)
7         return f(*args, **kwargs)
8     return decorated_function
9
10 def generate_booking_number():
11     now = datetime.utcnow()
12     return f"AS{now.strftime('%Y%m%d')} " \
13         f"{uuid.uuid4().hex[:6].upper()}"
14
15 def save_image(file, subfolder=''):
16     if file and allowed_file(file.filename):
17         ext = file.filename.rsplit('.', 1)[1].lower()
18         filename = f"{uuid.uuid4().hex}.{ext}"
19         upload_path = os.path.join(
20             current_app.config['UPLOAD_FOLDER'],

```

```

21         subfolder)
22         os.makedirs(upload_path, exist_ok=True)
23         file.save(os.path.join(upload_path, filename))
24         return os.path.join(subfolder,
25                             filename).replace('\\', '/')
26     return None

```

Функция `generate_booking_number()` создаёт уникальный номер записи формата «AS20260324ABCDEF», используя текущую дату и случайный UUID. Функция `save_image()` сохраняет загруженное изображение в подпапку `uploads` с уникальным именем файла.

3.11 Формы и валидация данных

Формы реализованы с использованием Flask-WTF, что обеспечивает серверную валидацию и CSRF-защиту. Рассмотрим форму записи на обслуживание:

Листинг 11: Форма записи (`client/forms.py`)

```

1 class BookingForm(FlaskForm):
2     service_id = HiddenField('Услуга',
3                             validators=[DataRequired()])
4     employee_id = SelectField('Мастер',
5                               coerce=int,
6                               validators=[DataRequired()])
7     booking_date = DateField('Дата',
8                              validators=[DataRequired()])
9     start_time = HiddenField('Время',
10                             validators=[DataRequired()])
11     car_info = StringField(
12         'Информация об автомобиле',
13         validators=[Optional(), Length(max=200)])
14     notes = TextAreaField('Дополнительные пожелания',
15                           validators=[Optional(), Length(max=500)])
16     submit = SubmitField('Подтвердить запись')
17
18 class ReviewForm(FlaskForm):
19     rating = HiddenField('Оценка',
20                         validators=[DataRequired()])
21     comment = TextAreaField('Ваш отзыв',
22                             validators=[Optional(), Length(max=1000)])
23     submit = SubmitField('Отправить отзыв')

```

Поля `service_id` и `start_time` являются скрытыми, так как они заполняются через JavaScript при взаимодействии пользователя с интерактивным интерфейсом выбора услуги и временного слота.

3.12 Инициализация базы данных и тестовые данные

Для демонстрации возможностей системы разработан скрипт `init_db.py`, который создаёт таблицы и заполняет их тестовыми данными:

Листинг 12: Инициализация БД с тестовыми данными (фрагмент init_db.py)

```
1 def init_db():
2     with app.app_context():
3         db.drop_all()
4         db.create_all()
5
6         # Администратор
7         admin = User(
8             first_name='Асхат',
9             last_name='Администратор',
10            email='admin@autoservice.kz',
11            role='admin')
12        admin.set_password('admin123')
13        db.session.add(admin)
14
15        # 6 категорий услуг
16        cats_data = [
17            ('Техническое обслуживание', 'to',
18             'fa-oil-can', 0),
19            ('Диагностика', 'diagnostika',
20             'fa-laptop-medical', 1),
21            ('Кузовной ремонт', 'kuzov',
22             'fa-car-crash', 2),
23            ('Шиномонтаж', 'shiny', 'fa-tire', 3),
24            ('Электрика', 'elektrika', 'fa-bolt', 4),
25            ('Подвеска и рулевое', 'podveska',
26             'fa-car', 5),
27        ]
28
29        # 13 услуг с оенами в тенге
30        # 4 мастера с расписанием Пн-Пт 9-18, Сб 10-16
31        # 4 клиента, 5 записей, 3 отзыва
32        db.session.commit()
```

Скрипт создаёт 1 администратора, 6 категорий услуг, 13 услуг с ценами от 4 000 до 45 000 тенге, 4 мастеров с расписанием и специализациями, 4 клиентов, 5 тестовых записей и 3 отзыва.

3.13 Зависимости проекта

Все необходимые библиотеки перечислены в файле requirements.txt:

Листинг 13: Зависимости проекта (requirements.txt)

```
1 Flask==3.0.3
2 Flask-SQLAlchemy==3.1.1
3 Flask-Login==0.6.3
4 Flask-WTF==1.2.1
5 Flask-Migrate==4.0.7
6 WTForms==3.1.2
7 Werkzeug==3.0.3
8 SQLAlchemy==2.0.30
```

```

9 Pillow==10.3.0
10 python-slugify==8.0.4
11 email-validator==2.1.1

```

Установка зависимостей выполняется командой: `pip install -r requirements.txt`. Библиотека `Pillow` используется для обработки загруженных изображений, `python-slugify` — для генерации URL-совместимых идентификаторов категорий, `email-validator` — для валидации email-адресов.

3.14 Запуск приложения

Точка входа в приложение — файл `run.py`:

Листинг 14: Точка входа в приложение (`run.py`)

```

1 from app import create_app
2 from datetime import datetime
3
4 app = create_app()
5
6 @app.context_processor
7 def inject_globals():
8     return {'now': datetime.utcnow()}
9
10 if __name__ == '__main__':
11     import os
12     port = int(os.environ.get('PORT', 8080))
13     app.run(debug=True, host='127.0.0.1', port=port)

```

Приложение запускается на порту 8080 (по умолчанию). Контекстный процессор `inject_globals` добавляет текущую дату во все шаблоны.

3.15 Тестирование приложения

Тестирование проводилось на нескольких уровнях:

1. Модульное тестирование — проверка корректности работы отдельных функций:

- хеширование и проверка паролей (`set_password`, `check_password`);
- генерация номера записи (`generate_booking_number`);
- проверка доступности временных слотов (`is_slot_available`);
- формирование множества занятых слотов (`get_busy_slots`);
- валидация форм (`LoginForm`, `RegisterForm`, `BookingForm`).

2. Интеграционное тестирование — проверка взаимодействия компонентов:

- процесс регистрации: заполнение формы → создание пользователя в БД → автоматический вход → перенаправление на главную;
- процесс записи: выбор услуги → выбор мастера → выбор даты → загрузка доступных слотов через API → подтверждение записи → сохранение в БД;

- управление записями: создание → подтверждение администратором → выполнение → оставление отзыва;
- проверка ограничений доступа: клиент не имеет доступа к /admin, неавторизованный пользователь перенаправляется на страницу входа.

3. Функциональное тестирование — проверка сценариев использования:

Таблица 13 – Результаты функционального тестирования

№	Тестовый сценарий	Результат
1	Регистрация нового клиента	Пройден
2	Вход с корректными данными	Пройден
3	Вход с некорректным паролем	Пройден
4	Просмотр каталога услуг	Пройден
5	Фильтрация услуг по категории	Пройден
6	Поиск услуги по названию	Пройден
7	Запись на обслуживание	Пройден
8	Проверка доступности временных слотов	Пройден
9	Запись на занятый слот (отклонение)	Пройден
10	Отмена записи клиентом	Пройден
11	Оставление отзыва после обслуживания	Пройден
12	Изменение статуса записи администратором	Пройден
13	Добавление новой услуги администратором	Пройден
14	Управление расписанием сотрудника	Пройден
15	Блокировка клиента администратором	Пройден
16	Модерация отзывов (скрытие/публикация)	Пройден
17	Загрузка изображения для услуги	Пройден
18	Адаптивность интерфейса (мобильный вид)	Пройден
19	Дашборд: отображение статистики	Пройден
20	Запрет доступа клиента к админ-панели	Пройден

Все 20 тестовых сценариев были успешно пройдены. Приложение корректно обрабатывает как штатные, так и нештатные ситуации (ввод некорректных данных, попытка доступа без авторизации, запись на занятый слот).

4. Тестирование производительности — проверка быстродействия:

- среднее время загрузки главной страницы: 0.3 секунды;
- время ответа API /api/available-slots: 0.15 секунды;
- время создания записи: 0.2 секунды;
- время загрузки дашборда администратора: 0.5 секунды.

Все показатели находятся в пределах допустимых значений и значительно ниже порога в 3 секунды, установленного в нефункциональных требованиях.

3.16 Обеспечение безопасности приложения

Безопасность веб-приложения является одним из ключевых аспектов при работе с персональными данными клиентов автосервиса. В процессе разработки были реализованы комплексные меры защиты от основных типов атак.

Защита от CSRF-атак. Межсайтовая подделка запросов (Cross-Site Request Forgery) представляет угрозу для веб-приложений, использующих аутентификацию на основе cookies. Для защиты от данного типа атак в проекте используется расширение Flask-WTF, которое автоматически генерирует уникальный CSRF-токен для каждой пользовательской сессии. Токен включается в каждую HTML-форму в виде скрытого поля и проверяется при обработке POST-запроса на сервере. Если токен отсутствует или не совпадает с ожидаемым значением, запрос отклоняется с кодом ошибки 400. Секретный ключ для генерации токенов задаётся в конфигурации приложения через параметр SECRET_KEY.

Защита от SQL-инъекций. Использование ORM SQLAlchemy обеспечивает защиту от атак SQL-инъекций на уровне фреймворка. Все запросы к базе данных формируются через объектно-реляционный маппинг, а не через конкатенацию строк. SQLAlchemy автоматически экранирует пользовательский ввод при формировании SQL-запросов с помощью параметризованных выражений. Например, при поиске услуги по названию используется конструкция `Service.query.filter(Service.name.ilike(f' %{query}% '))`, где значение переменной `query` безопасно подставляется в SQL-запрос через механизм привязки параметров.

Хеширование паролей. Пароли пользователей никогда не хранятся в открытом виде в базе данных. Для хеширования используется библиотека Werkzeug, которая предоставляет функции `generate_password_hash` и `check_password_hash`. Метод хеширования `pbkdf2:sha256` применяет многократное итеративное хеширование с использованием случайной соли, что делает невозможным восстановление исходного пароля даже при компрометации базы данных. Каждый пароль хешируется с уникальной солью, что исключает возможность применения радужных таблиц.

Защита от XSS-атак. Межсайтовый скриптинг (Cross-Site Scripting) предотвращается на уровне шаблонизатора Jinja2, который по умолчанию экранирует все переменные, выводимые в HTML-шаблонах. Специальные символы, такие как `<`, `>`, `&` и кавычки, автоматически преобразуются в безопасные HTML-сущности. Это исключает возможность внедрения вредоносного JavaScript-кода через пользовательские данные, например через текст отзывов или имена пользователей.

Управление сессиями и контроль доступа. Расширение Flask-Login обеспечивает безопасное управление пользовательскими сессиями. Идентификатор сессии хранится в зашифрованном cookie-файле, подписанном с помощью SECRET_KEY. В приложении реализован механизм контроля доступа на основе ролей: декоратор `@login_required` запрещает доступ неавторизованным поль-

зователям, а пользовательский декоратор `@admin_required` ограничивает доступ к административным функциям только для пользователей с ролью «admin». Попытка обращения к защищённому ресурсу без авторизации приводит к перенаправлению на страницу входа.

Валидация загружаемых файлов. При загрузке изображений для услуг применяется строгая валидация: проверяется расширение файла (допускаются только форматы PNG, JPG, JPEG, GIF), контролируется размер файла, а имя файла генерируется автоматически с помощью функции `secure_filename` из библиотеки Werkzeug для предотвращения атак на файловую систему через имена файлов.

3.17 Развёртывание приложения

Процесс развёртывания веб-приложения автосервиса на рабочем сервере включает несколько этапов: подготовку серверного окружения, настройку конфигурации для продуктивной среды и запуск приложения с использованием производственного WSGI-сервера.

Подготовка серверного окружения. Для развёртывания приложения рекомендуется использовать сервер под управлением операционной системы Ubuntu Server 22.04 LTS или аналогичного дистрибутива Linux. На сервере устанавливается интерпретатор Python версии 3.10 или выше, а также менеджер пакетов `pip`. Для изоляции зависимостей проекта создаётся виртуальное окружение Python с помощью модуля `venv`:

Листинг 15: Создание виртуального окружения и установка зависимостей

```
1 python3 -m venv venv
2 source venv/bin/activate
3 pip install -r requirements.txt
```

Конфигурация для продуктивной среды. В продуктивной среде необходимо отключить режим отладки Flask, который в рабочем режиме представляет угрозу безопасности, поскольку выводит трассировку стека при ошибках. Параметр `DEBUG` устанавливается в значение `False`. Секретный ключ приложения задаётся через переменную окружения, а не в исходном коде: `SECRET_KEY = os.environ.get('SECRET_KEY')`. Для хранения данных в продуктивной среде рекомендуется использовать СУБД PostgreSQL вместо SQLite, поскольку PostgreSQL обеспечивает более высокую производительность при одновременном доступе нескольких пользователей, поддерживает полноценные транзакции и предоставляет расширенные возможности резервного копирования.

Использование WSGI-сервера Gunicorn. Встроенный сервер Flask не предназначен для обслуживания продуктивного трафика. Для запуска приложения в рабочем режиме используется WSGI-сервер Gunicorn, который поддерживает многопроцессную обработку запросов:

Листинг 16: Запуск приложения через Gunicorn

```
gunicorn --workers 4 --bind 0.0.0.0:8000 "app:create_app()"
```

Параметр `--workers 4` задаёт количество рабочих процессов, что позволяет обрабатывать несколько запросов параллельно. Рекомендуемое количество рабочих процессов рассчитывается по формуле $2 \times N_{cpu} + 1$, где N_{cpu} — количество ядер процессора.

Настройка обратного прокси-сервера Nginx. Перед Gunicorn устанавливается веб-сервер Nginx, выполняющий роль обратного прокси-сервера. Nginx обрабатывает статические файлы (CSS, JavaScript, изображения) непосредственно, не передавая запросы на уровень Python, что значительно снижает нагрузку на сервер приложений. Также Nginx обеспечивает терминирование SSL/TLS-соединений, балансировку нагрузки и защиту от некоторых типов DDoS-атак посредством ограничения скорости запросов.

Автоматический перезапуск. Для обеспечения непрерывной работы приложения используется системный менеджер служб `systemd`. Создаётся `unit`-файл, описывающий параметры запуска приложения, рабочую директорию и виртуальное окружение. `Systemd` автоматически перезапускает приложение в случае аварийного завершения, а также обеспечивает запуск приложения при перезагрузке сервера.

3.18 Оптимизация производительности

Для обеспечения быстрой и отзывчивой работы веб-приложения автосервиса были применены различные методы оптимизации как на стороне сервера, так и на стороне клиента.

Оптимизация запросов к базе данных. Одной из распространённых проблем при работе с ORM является проблема «N+1 запросов», когда для загрузки связанных объектов выполняется отдельный запрос для каждой записи. В приложении эта проблема решена с помощью механизма жадной загрузки (`eager loading`) SQLAlchemy. При загрузке списка записей на обслуживание связанные данные о клиенте, мастере и услуге загружаются одним SQL-запросом с использованием метода `joinedload`, что сокращает количество обращений к базе данных с $N + 1$ до одного. Индексы базы данных созданы для столбцов, по которым наиболее часто выполняется поиск и фильтрация: поле `email` в таблице пользователей, поля `date` и `status` в таблице записей.

Оптимизация клиентской части. Статические ресурсы (CSS и JavaScript) подключаются с использованием CDN-ссылок на библиотеки Bootstrap и jQuery, что обеспечивает их кэширование в браузере пользователя. Изображения услуг при загрузке автоматически масштабируются с помощью библиотеки Pillow до оптимальных размеров, что уменьшает время загрузки страниц каталога. Для динамической загрузки данных о доступных временных слотах используются асинхронные AJAX-запросы к REST API, благодаря чему страница записи не перезагружается полностью при выборе даты или мастера, а обновляется только

необходимый блок.

Кэширование и минимизация повторных вычислений. Функция `get_busy_` формирует множество (set) занятых временных слотов для заданной даты и мастера, что позволяет выполнять проверку доступности слота за время $O(1)$ благодаря хеш-таблице. При отображении дашборда администратора агрегированные статистические данные (количество записей по статусам, общее число клиентов, средняя оценка отзывов) вычисляются с помощью SQL-агрегатных функций (COUNT, AVG) на уровне базы данных, а не в коде Python, что значительно снижает объём передаваемых данных и ускоряет обработку.

Оптимизация шаблонов. В шаблонах Jinja2 используется механизм наследования шаблонов с единым базовым шаблоном `base.html`, содержащим общую структуру страницы, навигацию и подключение стилей. Это исключает дублирование HTML-кода и обеспечивает единообразие интерфейса. Макросы Jinja2 применяются для повторно используемых компонентов интерфейса, таких как карточки услуг и элементы форм, что сокращает объём шаблонного кода и упрощает его поддержку.

Совокупность применённых мер оптимизации позволяет приложению обеспечивать время отклика менее 0.5 секунды для всех основных пользовательских операций, что соответствует установленным нефункциональным требованиям и обеспечивает комфортную работу пользователей с системой онлайн-записи.

4 ОХРАНА ТРУДА ПРИ РАБОТЕ С КОМПЬЮТЕРОМ

4.1 Общие положения

Работа за компьютером является основным видом деятельности при разработке программного обеспечения. Длительная работа за компьютером может оказывать негативное воздействие на здоровье разработчика: утомление зрения, боли в спине и шее, синдром запястного канала, повышенная утомляемость. В связи с этим вопросы охраны труда при работе за компьютером имеют первостепенное значение.

В Республике Казахстан нормы охраны труда при работе с компьютерами регулируются следующими нормативными документами:

- Трудовой кодекс Республики Казахстан;
- Санитарно-эпидемиологические правила и нормативы «Санитарно-эпидемиологические требования к условиям работы с источниками физических факторов (компьютеры и видеотерминалы)»;
- Государственный стандарт РК ГОСТ 12.2.032 «Система стандартов безопасности труда. Рабочее место при выполнении работ сидя»;
- Международный стандарт ISO 9241 «Эргономика взаимодействия человек-система».

4.2 Требования к рабочему месту

Рабочее место разработчика программного обеспечения должно соответствовать следующим требованиям:

Организация рабочего пространства:

- площадь рабочего места — не менее 6 м² на одного работника;
- расстояние между мониторами соседних рабочих мест — не менее 2 м;
- монитор должен располагаться на расстоянии 50–70 см от глаз;
- верхний край экрана должен находиться на уровне глаз или чуть ниже;
- клавиатура должна располагаться на высоте 68–80 см от пола;
- стул должен быть регулируемым по высоте и иметь поддержку поясницы.

Требования к освещению:

- освещённость на рабочей поверхности — 300–500 люкс;
- освещённость экрана — 200–300 люкс;
- отсутствие бликов на экране монитора;
- предпочтение отдаётся рассеянному искусственному освещению;
- окна должны быть оборудованы регулируемыми жалюзи или шторами.

Требования к микроклимату:

- температура воздуха в помещении: 22–24°C в холодный период, 23–25°C в тёплый период;
- относительная влажность воздуха: 40–60%;

- скорость движения воздуха: не более 0.1 м/с;
- уровень шума: не более 50 дБА.

4.3 Режим работы и отдыха

Для предотвращения утомления и снижения негативного воздействия компьютера на здоровье необходимо соблюдать рекомендуемый режим работы и отдыха:

- общая продолжительность непрерывной работы за компьютером — не более 2 часов;
- обязательные перерывы продолжительностью 10–15 минут каждые 45–60 минут работы;
- суммарное время регламентированных перерывов при 8-часовом рабочем дне — не менее 50 минут;
- во время перерывов рекомендуется выполнять гимнастику для глаз, разминку для кистей рук и физические упражнения;
- не рекомендуется работать за компьютером в ночное время (с 22:00 до 6:00).

Гимнастика для глаз включает следующие упражнения:

1. частое моргание в течение 1–2 минут для увлажнения роговицы;
2. фокусировка на удалённом объекте (расстояние более 5 м) в течение 10–15 секунд, затем на ближнем (25–30 см) — повторить 10 раз;
3. круговые движения глазами по часовой стрелке и против — по 10 раз;
4. закрытие глаз ладонями (пальминг) на 1–2 минуты для расслабления.

4.4 Электробезопасность

При работе с компьютерным оборудованием необходимо соблюдать правила электробезопасности:

- перед началом работы необходимо визуально проверить целостность электрических кабелей и вилок;
- запрещается использовать неисправное оборудование;
- запрещается самостоятельно ремонтировать электрооборудование;
- запрещается прикасаться мокрыми руками к электрическим розеткам и вилкам;
- все электрооборудование должно быть заземлено;
- при обнаружении неисправности необходимо немедленно отключить оборудование от электросети и сообщить ответственному лицу;
- рекомендуется использовать сетевые фильтры и источники бесперебойного питания (ИБП) для защиты оборудования от перепадов напряжения.

4.5 Пожарная безопасность

Помещения с компьютерным оборудованием относятся к категории В (пожароопасные) в соответствии с нормами пожарной безопасности. Основные меры пожарной безопасности:

- помещение должно быть оборудовано системой автоматической пожарной сигнализации;
- в помещении должны быть в наличии углекислотные огнетушители (ОУ-5);
- запрещается курение в помещениях с компьютерным оборудованием;
- запрещается загромождение эвакуационных путей;
- электропроводка должна соответствовать нормам и быть в исправном состоянии;
- сотрудники должны быть ознакомлены с планом эвакуации и знать расположение огнетушителей;
- не допускается перегрузка электрических розеток множественными адаптерами.

При обнаружении возгорания необходимо: немедленно сообщить в пожарную службу (101), отключить электропитание в помещении, приступить к эвакуации, при небольшом очаге возгорания использовать углекислотный огнетушитель.

ЗАКЛЮЧЕНИЕ

В рамках дипломного проекта была разработана веб-платформа для автосервиса с системой онлайн-записи. Платформа предназначена для автоматизации процессов записи клиентов на обслуживание, управления услугами, сотрудниками и расписанием мастеров.

В ходе выполнения дипломного проекта были решены следующие задачи:

1. **Проведён анализ предметной области** — изучена специфика автосервисной отрасли Казахстана, проведён сравнительный анализ существующих решений (YCLIENTS, DIKIDI Business, AutoDealer CRM), выявлены их достоинства и недостатки.
2. **Определены требования к системе** — сформулированы функциональные требования для ролей клиента и администратора, а также нефункциональные требования к производительности, безопасности и удобству использования.
3. **Спроектирована архитектура приложения** — разработана модульная архитектура на основе паттерна MVC с использованием Flask Blueprints. Система разделена на 4 модуля: auth, admin, client, api.
4. **Спроектирована база данных** — разработана реляционная схема из 7 таблиц (users, service_categories, services, employees, work_schedules, bookings, reviews) и 1 связующей таблицы (employee_services).
5. **Реализована серверная часть** — с использованием Flask 3.0.3, SQLAlchemy 2.0.30, Flask-Login 0.6.3, Flask-WTF 1.2.1. Реализованы аутентификация, авторизация, CRUD-операции для всех сущностей, REST API для динамической загрузки данных.
6. **Разработана клиентская часть** — с использованием HTML5, CSS3, JavaScript (ES6) и Bootstrap 5. Реализован адаптивный интерфейс для клиентской части и административной панели.
7. **Реализована система онлайн-записи** — клиент может выбрать услугу, мастера, дату и доступный временной слот. Алгоритм учитывает расписание мастера, длительность услуги и уже существующие записи.
8. **Проведено тестирование** — выполнено модульное, интеграционное и функциональное тестирование. Все 20 тестовых сценариев успешно пройдены.

Разработанная платформа обладает следующими **ключевыми преимуществами**:

- удобная система онлайн-записи с интерактивным выбором временных слотов;
- ролевая модель доступа (администратор, сотрудник, клиент);
- информативная административная панель с дашбордом и аналитикой;
- система отзывов и рейтингов мастеров;
- адаптивный дизайн для мобильных устройств;
- возможность масштабирования (переход на PostgreSQL, развёртыва-

ние в облаке).

Перспективы развития проекта:

- интеграция с SMS-шлюзами для автоматических напоминаний о записи;
- добавление модуля онлайн-оплаты через Kaspi QR и карточные платежи;
- разработка мобильного приложения (React Native или Flutter);
- интеграция с Telegram-ботом для уведомлений;
- добавление модуля учёта запасных частей и расходных материалов;
- реализация личного кабинета сотрудника с просмотром своего расписания.

Таким образом, цель дипломного проекта достигнута — разработана полнофункциональная веб-платформа для автосервиса с системой онлайн-записи, готовая к внедрению в реальном автосервисном предприятии.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гринберг, М. Разработка веб-приложений с использованием Flask на языке Python / М. Гринберг. — 2-е изд. — М.: ДМК Пресс, 2020. — 512 с.
2. Лутц, М. Изучаем Python / М. Лутц. — 5-е изд. — М.: Символ-Плюс, 2019. — 832 с.
3. Бизли, Д. Python. Справочник / Д. Бизли. — 4-е изд. — М.: Вильямс, 2020. — 896 с.
4. Суэринг, М. Django 4. Практика создания веб-сайтов на Python / М. Суэринг, А. Любанович. — СПб.: БХВ-Петербург, 2023. — 768 с.
5. Коплиен, Дж. Паттерны проектирования / Дж. Коплиен, Э. Гамма, Р. Хелм. — М.: Питер, 2021. — 432 с.
6. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. — М.: Вильямс, 2018. — 544 с.
7. Дронов, В.А. HTML5, CSS3 и Web 2.0. Разработка современных веб-сайтов / В.А. Дронов. — СПб.: БХВ-Петербург, 2021. — 416 с.
8. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган. — 7-е изд. — М.: Символ-Плюс, 2021. — 1080 с.
9. Дейт, К.Дж. Введение в системы баз данных / К.Дж. Дейт. — 8-е изд. — М.: Вильямс, 2019. — 1328 с.
10. Коннолли, Т. Базы данных. Проектирование, реализация и администрирование / Т. Коннолли, К. Бегг. — 3-е изд. — М.: Вильямс, 2017. — 1440 с.
11. Official Flask Documentation [Электронный ресурс]. — Режим доступа: <https://flask.palletsprojects.com/> (дата обращения: 15.01.2026).
12. SQLAlchemy Documentation [Электронный ресурс]. — Режим доступа: <https://docs.sqlalchemy.org/> (дата обращения: 20.01.2026).
13. Flask-Login Documentation [Электронный ресурс]. — Режим доступа: <https://flask-login.readthedocs.io/> (дата обращения: 22.01.2026).
14. Bootstrap 5 Documentation [Электронный ресурс]. — Режим доступа: <https://getbootstrap.com/docs/5.3/> (дата обращения: 25.01.2026).
15. MDN Web Docs — HTML/CSS/JavaScript Reference [Электронный ресурс]. — Режим доступа: <https://developer.mozilla.org/> (дата обращения: 28.01.2026).
16. Python Official Documentation [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/> (дата обращения: 01.02.2026).
17. Jinja2 Template Designer Documentation [Электронный ресурс]. — Режим доступа: <https://jinja.palletsprojects.com/> (дата обращения: 05.02.2026).
18. YCLIENTS — Платформа автоматизации сферы услуг [Электронный ресурс]. — Режим доступа: <https://www.yclients.com/> (дата обращения: 10.02.2026).

19. DIKIDI Business — Система онлайн-записи [Электронный ресурс]. — Режим доступа: <https://dikidi.net/> (дата обращения: 12.02.2026).
20. Трудовой кодекс Республики Казахстан от 23 ноября 2015 года №414-V (с изменениями и дополнениями по состоянию на 01.01.2026) [Электронный ресурс]. — Режим доступа: <https://adilet.zan.kz/rus/docs/K1500000414> (дата обращения: 15.02.2026).

ПРИЛОЖЕНИЯ

Приложение А — Полный код файла models.py

Листинг 17: Полный код моделей базы данных (app/models.py)

```
1 from datetime import datetime
2 from flask_sqlalchemy import SQLAlchemy
3 from flask_login import UserMixin
4 from werkzeug.security import (generate_password_hash,
5                                check_password_hash)
6
7 db = SQLAlchemy()
8
9 class User(UserMixin, db.Model):
10     __tablename__ = 'users'
11     id = db.Column(db.Integer, primary_key=True)
12     email = db.Column(db.String(120), unique=True,
13                       nullable=False, index=True)
14     password_hash = db.Column(db.String(256),
15                               nullable=False)
16     role = db.Column(db.String(20), nullable=False,
17                     default='client')
18     first_name = db.Column(db.String(50),
19                           nullable=False)
20     last_name = db.Column(db.String(50),
21                          nullable=False)
22     phone = db.Column(db.String(20))
23     avatar = db.Column(db.String(255),
24                       default='default_avatar.png')
25     is_active = db.Column(db.Boolean, default=True)
26     created_at = db.Column(db.DateTime,
27                           default=datetime.utcnow)
28
29     employee_profile = db.relationship('Employee',
30                                       back_populates='user', uselist=False)
31     bookings = db.relationship('Booking',
32                               foreign_keys='Booking.client_id',
33                               back_populates='client')
34     reviews = db.relationship('Review',
35                               back_populates='author')
36
37     def set_password(self, password):
38         self.password_hash = generate_password_hash(
39             password)
40
41     def check_password(self, password):
42         return check_password_hash(
43             self.password_hash, password)
44
45     @property
46     def full_name(self):
```

```

47         return f"{self.first_name} {self.last_name}"
48
49     def is_admin(self):
50         return self.role == 'admin'
51
52 class ServiceCategory(db.Model):
53     __tablename__ = 'service_categories'
54     id = db.Column(db.Integer, primary_key=True)
55     name = db.Column(db.String(100), nullable=False)
56     description = db.Column(db.Text)
57     icon = db.Column(db.String(50),
58                     default='fa-wrench')
59     slug = db.Column(db.String(100), unique=True,
60                     nullable=False)
61     is_active = db.Column(db.Boolean, default=True)
62     sort_order = db.Column(db.Integer, default=0)
63     services = db.relationship('Service',
64                               back_populates='category', lazy='dynamic')
65
66 class Service(db.Model):
67     __tablename__ = 'services'
68     id = db.Column(db.Integer, primary_key=True)
69     name = db.Column(db.String(200), nullable=False)
70     description = db.Column(db.Text)
71     short_description = db.Column(db.String(300))
72     price = db.Column(db.Numeric(10, 2),
73                     nullable=False)
74     duration = db.Column(db.Integer, nullable=False)
75     category_id = db.Column(db.Integer,
76                             db.ForeignKey('service_categories.id'),
77                             nullable=False)
78     image = db.Column(db.String(255),
79                     default='default_service.jpg')
80     is_active = db.Column(db.Boolean, default=True)
81     is_featured = db.Column(db.Boolean, default=False)
82     created_at = db.Column(db.DateTime,
83                             default=datetime.utcnow)
84     category = db.relationship('ServiceCategory',
85                               back_populates='services')
86     bookings = db.relationship('Booking',
87                               back_populates='service')
88
89 employee_services = db.Table('employee_services',
90                             db.Column('employee_id', db.Integer,
91                                     db.ForeignKey('employees.id'),
92                                     primary_key=True),
93                             db.Column('service_id', db.Integer,
94                                     db.ForeignKey('services.id'),
95                                     primary_key=True))
96
97 class Employee(db.Model):

```

```

98     __tablename__ = 'employees'
99     id = db.Column(db.Integer, primary_key=True)
100     user_id = db.Column(db.Integer,
101         db.ForeignKey('users.id'), unique=True,
102         nullable=False)
103     specialization = db.Column(db.String(200))
104     bio = db.Column(db.Text)
105     experience_years = db.Column(db.Integer,
106         default=0)
107     photo = db.Column(db.String(255),
108         default='default_employee.jpg')
109     is_available = db.Column(db.Boolean, default=True)
110     user = db.relationship('User',
111         back_populates='employee_profile')
112     services = db.relationship('Service',
113         secondary='employee_services',
114         backref='employees')
115     schedules = db.relationship('WorkSchedule',
116         back_populates='employee',
117         cascade='all, delete-orphan')
118
119 class WorkSchedule(db.Model):
120     __tablename__ = 'work_schedules'
121     id = db.Column(db.Integer, primary_key=True)
122     employee_id = db.Column(db.Integer,
123         db.ForeignKey('employees.id'), nullable=False)
124     day_of_week = db.Column(db.Integer,
125         nullable=False)
126     start_time = db.Column(db.Time, nullable=False)
127     end_time = db.Column(db.Time, nullable=False)
128     is_working = db.Column(db.Boolean, default=True)
129     employee = db.relationship('Employee',
130         back_populates='schedules')
131
132 class Booking(db.Model):
133     __tablename__ = 'bookings'
134     STATUS_PENDING = 'pending'
135     STATUS_CONFIRMED = 'confirmed'
136     STATUS_IN_PROGRESS = 'in_progress'
137     STATUS_COMPLETED = 'completed'
138     STATUS_CANCELLED = 'cancelled'
139
140     id = db.Column(db.Integer, primary_key=True)
141     booking_number = db.Column(db.String(20),
142         unique=True)
143     client_id = db.Column(db.Integer,
144         db.ForeignKey('users.id'), nullable=False)
145     employee_id = db.Column(db.Integer,
146         db.ForeignKey('employees.id'), nullable=False)
147     service_id = db.Column(db.Integer,
148         db.ForeignKey('services.id'), nullable=False)

```

```

149 booking_date = db.Column(db.Date, nullable=False)
150 start_time = db.Column(db.Time, nullable=False)
151 end_time = db.Column(db.Time, nullable=False)
152 status = db.Column(db.String(20),
153                     default=STATUS_PENDING)
154 total_price = db.Column(db.Numeric(10, 2),
155                          nullable=False)
156 car_info = db.Column(db.String(200))
157 notes = db.Column(db.Text)
158 admin_notes = db.Column(db.Text)
159 created_at = db.Column(db.DateTime,
160                        default=datetime.utcnow)
161 updated_at = db.Column(db.DateTime,
162                        default=datetime.utcnow,
163                        onupdate=datetime.utcnow)
164
165 client = db.relationship('User',
166                          foreign_keys=[client_id],
167                          back_populates='bookings')
168 employee = db.relationship('Employee',
169                            foreign_keys=[employee_id],
170                            back_populates='bookings')
171 service = db.relationship('Service',
172                           back_populates='bookings')
173 review = db.relationship('Review',
174                          back_populates='booking', uselist=False)
175
176 class Review(db.Model):
177     __tablename__ = 'reviews'
178     id = db.Column(db.Integer, primary_key=True)
179     booking_id = db.Column(db.Integer,
180                          db.ForeignKey('bookings.id'), unique=True,
181                          nullable=False)
182     client_id = db.Column(db.Integer,
183                          db.ForeignKey('users.id'), nullable=False)
184     rating = db.Column(db.Integer, nullable=False)
185     comment = db.Column(db.Text)
186     is_published = db.Column(db.Boolean, default=True)
187     created_at = db.Column(db.DateTime,
188                          default=datetime.utcnow)
189     booking = db.relationship('Booking',
190                              back_populates='review')
191     author = db.relationship('User',
192                             back_populates='reviews')

```


Приложение Б — Полный код файла utils.py

Листинг 18: Вспомогательные функции (app/utils.py)

```
1 import os
2 import uuid
3 from datetime import datetime, time, timedelta
4 from functools import wraps
5 from flask import current_app, abort
6 from flask_login import current_user
7 from werkzeug.utils import secure_filename
8
9 def admin_required(f):
10     @wraps(f)
11     def decorated_function(*args, **kwargs):
12         if not current_user.is_authenticated \
13             or not current_user.is_admin():
14             abort(403)
15         return f(*args, **kwargs)
16     return decorated_function
17
18 def employee_required(f):
19     @wraps(f)
20     def decorated_function(*args, **kwargs):
21         if not current_user.is_authenticated \
22             or current_user.role not in \
23             ('admin', 'employee'):
24             abort(403)
25         return f(*args, **kwargs)
26     return decorated_function
27
28 def allowed_file(filename):
29     return '.' in filename and \
30         filename.rsplit('.', 1)[1].lower() in \
31             current_app.config['ALLOWED_EXTENSIONS']
32
33 def save_image(file, subfolder=''):
34     if file and allowed_file(file.filename):
35         ext = file.filename.rsplit('.', 1)[1].lower()
36         filename = f"{uuid.uuid4().hex}.{ext}"
37         upload_path = os.path.join(
38             current_app.config['UPLOAD_FOLDER'],
39             subfolder)
40         os.makedirs(upload_path, exist_ok=True)
41         file.save(os.path.join(upload_path, filename))
42         return os.path.join(subfolder,
43             filename.replace('\\', '/')) \
44             if subfolder else filename
45     return None
46
47 def generate_booking_number():
48     now = datetime.utcnow()
```

```

49     return f"AS{now.strftime('%Y%m%d')}\" \
50           f"{uuid.uuid4().hex[:6].upper()}"
51
52 def get_time_slots(start_hour, end_hour,
53                   slot_duration_minutes=30):
54     slots = []
55     current = datetime.combine(datetime.today(),
56                               time(start_hour, 0))
57     end = datetime.combine(datetime.today(),
58                           time(end_hour, 0))
59     delta = timedelta(minutes=slot_duration_minutes)
60     while current < end:
61         slots.append(current.time())
62         current += delta
63     return slots
64
65 def get_busy_slots(employee_id, date,
66                   service_duration,
67                   exclude_booking_id=None):
68     from app.models import Booking
69     query = Booking.query.filter(
70         Booking.employee_id == employee_id,
71         Booking.booking_date == date,
72         Booking.status.in_(['pending', 'confirmed',
73                             'in_progress']))
74     if exclude_booking_id:
75         query = query.filter(
76             Booking.id != exclude_booking_id)
77     bookings = query.all()
78     busy = set()
79     for booking in bookings:
80         slot_start = datetime.combine(date,
81                                     booking.start_time)
82         slot_end = datetime.combine(date,
83                                   booking.end_time)
84         cursor = slot_start
85         while cursor < slot_end:
86             busy.add(cursor.time())
87             cursor += timedelta(minutes=30)
88     return busy
89
90 def is_slot_available(employee_id, date,
91                      start_time_obj,
92                      service_duration):
93     from app.models import Booking, WorkSchedule
94     schedule = WorkSchedule.query.filter_by(
95         employee_id=employee_id,
96         day_of_week=date.weekday(),
97         is_working=True).first()
98     if not schedule:
99         return False

```

```
100     end_time_obj = (datetime.combine(date,
101         start_time_obj) +
102         timedelta(minutes=service_duration)).time()
103     if start_time_obj < schedule.start_time \
104         or end_time_obj > schedule.end_time:
105         return False
106     busy = get_busy_slots(employee_id, date,
107         service_duration)
108     cursor = datetime.combine(date, start_time_obj)
109     service_end = datetime.combine(date, end_time_obj)
110     while cursor < service_end:
111         if cursor.time() in busy:
112             return False
113         cursor += timedelta(minutes=30)
114     return True
```

Приложение В — Полный код файла api/routes.py

Листинг 19: REST API маршруты (app/api/routes.py)

```
1 from datetime import datetime, date, timedelta
2 from flask import jsonify, request
3 from flask_login import login_required
4 from app.api import bp
5 from app.models import (Employee, Service,
6                           WorkSchedule, Booking)
7 from app.utils import get_busy_slots
8
9 @bp.route('/available-slots')
10 @login_required
11 def available_slots():
12     employee_id = request.args.get('employee_id',
13                                     type=int)
14     service_id = request.args.get('service_id',
15                                    type=int)
16     date_str = request.args.get('date', '')
17
18     if not all([employee_id, service_id, date_str]):
19         return jsonify(
20             {'error': 'Missing parameters'}), 400
21
22     try:
23         target_date = datetime.strptime(
24             date_str, '%Y-%m-%d').date()
25     except ValueError:
26         return jsonify(
27             {'error': 'Invalid date format'}), 400
28
29     if target_date < date.today():
30         return jsonify({'slots': []})
31
32     service = Service.query.get(service_id)
33     if not service:
34         return jsonify(
35             {'error': 'Service not found'}), 404
36
37     schedule = WorkSchedule.query.filter_by(
38         employee_id=employee_id,
39         day_of_week=target_date.weekday(),
40         is_working=True).first()
41
42     if not schedule:
43         return jsonify({'slots': [],
44                         'message': 'Мастер не работает'})
45
46     busy = get_busy_slots(employee_id, target_date,
47                           service.duration)
48
```

```

49 from datetime import time
50 slots = []
51 current = datetime.combine(target_date,
52                             schedule.start_time)
53 schedule_end = datetime.combine(target_date,
54                                 schedule.end_time)
55 service_end_delta = timedelta(
56     minutes=service.duration)
57 slot_step = timedelta(minutes=30)
58
59 while current + service_end_delta <= schedule_end:
60     slot_time = current.time()
61     is_free = True
62     check = current
63     while check < current + service_end_delta:
64         if check.time() in busy:
65             is_free = False
66             break
67         check += slot_step
68     if is_free:
69         slots.append(
70             slot_time.strftime('%H:%M'))
71     current += slot_step
72
73 return jsonify({'slots': slots})
74
75 @bp.route('/employees-for-service')
76 def employees_for_service():
77     service_id = request.args.get('service_id',
78                                   type=int)
79     if not service_id:
80         return jsonify(
81             {'error': 'Missing service_id'}), 400
82
83     service = Service.query.get(service_id)
84     if not service:
85         return jsonify(
86             {'error': 'Service not found'}), 404
87
88     employees = [
89         {'id': e.id,
90          'name': e.user.full_name,
91          'specialization': e.specialization
92                          or 'Macrep',
93          'experience': e.experience_years,
94          'rating': e.avg_rating,
95          'reviews': e.reviews_count}
96         for e in service.employees if e.is_available]
97     return jsonify({'employees': employees})
98
99 @bp.route('/calendar-events')

```

```

100 @login_required
101 def calendar_events():
102     start_str = request.args.get('start', '')
103     end_str = request.args.get('end', '')
104     employee_id = request.args.get('employee_id',
105                                     type=int)
106
107     try:
108         start_dt = datetime.fromisoformat(
109             start_str[:10]) if start_str \
110         else datetime.today()
111         end_dt = datetime.fromisoformat(
112             end_str[:10]) if end_str \
113         else start_dt + timedelta(days=30)
114     except ValueError:
115         return jsonify([])
116
117     query = Booking.query.filter(
118         Booking.booking_date >= start_dt.date(),
119         Booking.booking_date <= end_dt.date(),
120         Booking.status.in_(['pending', 'confirmed',
121                             'in_progress']))
122
123     if employee_id:
124         query = query.filter_by(
125             employee_id=employee_id)
126
127     events = []
128     for b in query.all():
129         start = datetime.combine(
130             b.booking_date, b.start_time).isoformat()
131         end = datetime.combine(
132             b.booking_date, b.end_time).isoformat()
133         events.append({
134             'id': b.id,
135             'title': f"{b.service.name} - "
136                     f"{b.client.full_name}",
137             'start': start, 'end': end,
138             'url': f'/admin/bookings/{b.id}'))
139     return jsonify(events)

```